NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

Centre for Professional and Continuing Education

(SCTP) Advanced Professional Certificate

Data Science and AI



L10 Transformers & GenAI

Attention · LLMs · RAG · the architecture behind ChatGPT

Video intro: https://youtu.be/dJ2zUsszac4

Interactive html: https://su-ntu-ctp.github.io/6m-data-3.10-Transformers-Attention-GenAI/

Transformer paper: https://arxiv.org/abs/1706.03762
We used this in our coaching session for Gemini Notebook, if you still have it, try using video or audio overview combined with "eli5" to understand it

L10 The Explainer Anchoring the AI
https://www.youtube.com/watch?v=dJ2zUsszac4

The Language Factory - How Transformer Works
https://www.youtube.com/watch?v=UBUeH2CfQyE

Embedding visualization: https://projector.tensorflow.org/

Copyright © 2026 Nanyang Technological University. All rights reserved.

Join at

slido.com

#DS6310



Copyright © 2026 Nanyang Technological University. All rights reserved.

Where We Left Off

Search shipped. Marcus's final ask.

Marcus, end of L09:

"Can we build a shopping ASSISTANT? Customer asks 'I need a summer outfit for a beach holiday under £200' — it understands the question AND recommends actual products from our catalogue."

What's different today

- **Open the black box** — Where do embeddings come from? Attention + transformers.
- **Use an LLM** — Load a small model, tokenize, generate, sample, chat-format.
- **Build RAG end-to-end** — Combine L09 retrieval with LLM generation — the GenAI stack.
- **Module 3 closes** — By the end of today you'll have the modern ML toolkit, end to end.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Today's Three Questions

From attention to GenAI

1

What IS Attention? How does it work?

Each word looks at every other word and decides which ones matter for its meaning.

2

What is an LLM, mechanically?

A transformer trained to predict the next token. Generation is a one-token-at-a-time loop.

3

How do you USE an LLM on YOUR data?

RAG: retrieve relevant docs, stuff them in the prompt, let the LLM answer with grounding.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Why Attention?

The context problem in one example

Read these two sentences:

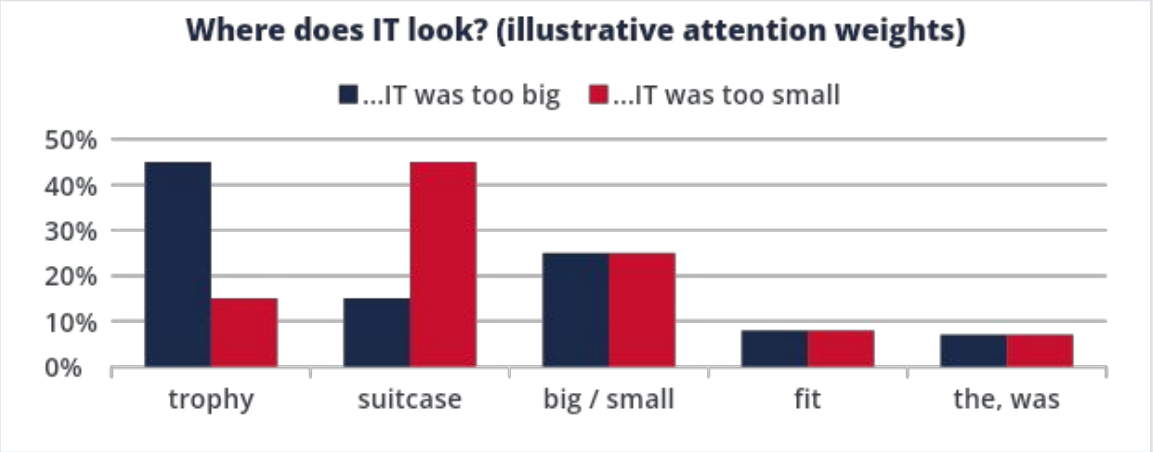
"The trophy didn't fit in the suitcase because IT was too big."

"The trophy didn't fit in the suitcase because IT was too small."

First IT = trophy (too big). Second IT = suitcase (too small). Same word, different meaning.

What the model needs: self-attention

- Each word looks at every other word in its OWN sentence (that's the "self")
- Weight key words heavily (trophy, big/small), filler lightly (the, was)
- Blend them into a context-aware meaning of IT



Copyright © 2026 Nanyang Technological University. All rights reserved.

Q · K · V — Three Jobs Per Word

A question, a name tag, an info card

Picture every word as a student in a classroom. Each one gets three jobs.

QUERY (Q)

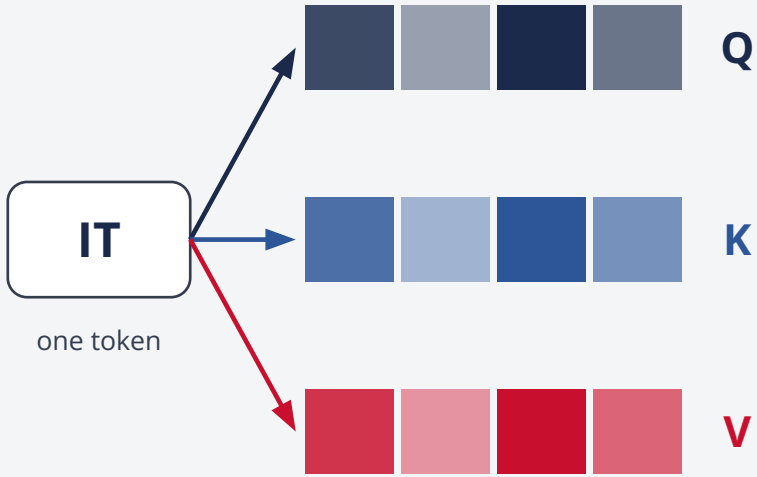
the question a word asks: “what am I looking for?”

KEY (K)

the name tag a word wears: “here’s what I am”

VALUE (V)

the info card a word holds: “here’s what I share”



So how does IT know it means “trophy”?

① ASK (IT’s Query)

IT asks the room: “which thing was too big?”

② MATCH (every Key)

“trophy” wears the best-matching name tag

③ BLEND (Values)

IT copies mostly trophy’s info card → IT = trophy

Copyright © 2026 Nanyang Technological University. All rights reserved.

How Attention Works

Three steps, in plain English

1

Match questions to name tags

Each word checks its question against every name tag.
Good match → high score. Poor match → low score.

2

Turn scores into percentages

Each word's scores become percentages that add up to 100%.
These percentages are the ATTENTION WEIGHTS.

3

Copy notes from the best matches

Each word copies notes from everyone's info cards — mostly from its best matches.
Result: every word now understands its context.

Attention weights (illustrative)

keys: the words being looked at

	the	cat	sat	on	mat
the	0.40	0.35	0.10	0.05	0.10
cat	0.15	0.45	0.25	0.05	0.10
sat	0.05	0.40	0.30	0.15	0.10
on	0.05	0.10	0.30	0.25	0.30
mat	0.10	0.15	0.15	0.25	0.35

Each row = one word asking its question. Each row sums to 1.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Where Q, K and V Come From

Three learned translators, then one match-and-blend calculation

① Each word makes its own Q, K and V

IT — **Embedding: [0.2, 0.7, 0.1, 0.5]**

the word's meaning as numbers (from L09)

× Query translator (W_Q)	→	**Q = [0.9, 0.1, 0.8]** "What am I looking for?"
× Key translator (W_K)	→	**K = [0.7, 0.2, 0.6]** "Here's what I am."
× Value translator (W_V)	→	**V = [0.4, 0.5, 0.3]** "Here's what I share."

The 3 translators are learned in training and shared by every word.

② IT uses them to work out its meaning

Word	Name tag (K)	Score	Weight
trophy	[0.8, 0.0, 0.9]	1.44	53%
suitcase	[0.6, 0.1, 0.5]	0.95	32%
the	[0.0, 0.9, 0.1]	0.17	15%

A · Match: IT's Q against each K ($Q \cdot K^T$)
Multiply position by position, then add:
0.9×0.8 + 0.1×0.0 + 0.8×0.9 = 1.44 (trophy)

B · Scores become percentages (softmax)
1.44, 0.95, 0.17 → 53%, 32%, 15% (adds to 100%)

C · Blend the info cards (V)
New IT = 53% trophy's V + 32% suitcase's V
+ 15% the's V → IT leans to "trophy"

Copyright © 2026 Nanyang Technological University. All rights reserved.

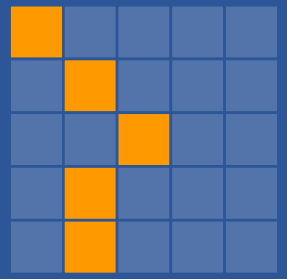
Multi-Head Attention

Several attentions in parallel, each with its specialty

One attention layer learns ONE relationship. Real models need many.

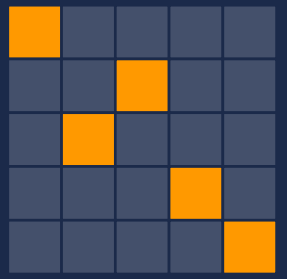
Head 1

Coreference
(pronouns → entities)



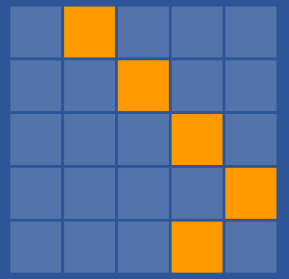
Head 2

Subject-verb
syntactic structure



Head 3

Modifier-noun
adjective relationships



Head 4

Long-range
discourse structure



Real transformers typically use 12-16 heads per layer × 12-96 layers.

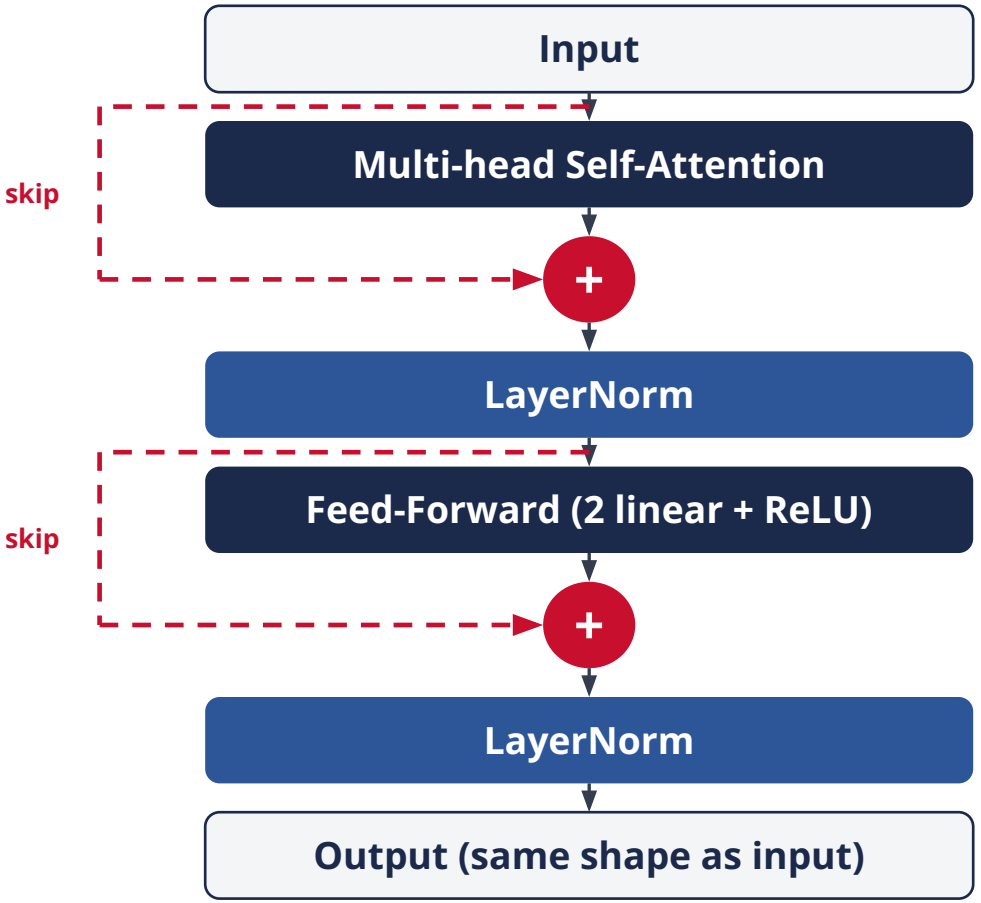
Each head's specialty EMERGES from training — nobody hand-designs them.

Copyright © 2026 Nanyang Technological University. All rights reserved.

The Transformer Block

The Lego brick of modern NLP

ONE block = attention + feed-forward, each wrapped in residual + LayerNorm



Attention = the meeting
Every word shares information with every other word.

Feed-forward = desk time
Each word then thinks on its own.

+ Residual = keep your notes
Add the original input back in, so nothing gets lost in deep stacks.

LayerNorm = volume limiter
Rescale to keep the numbers in a sensible range.

Output shape = Input shape
So blocks stack like Lego: 6 in MiniLM, 96 in GPT-3.

Copyright © 2026 Nanyang Technological University. All rights reserved.

How the Pieces Fit Together

One classroom: from a single question to a whole term

TRANSFORMER = the whole term — **6 to 96 blocks, stacked**

TRANSFORMER BLOCK = one class session

MULTI-HEAD ATTENTION = group discussions at once

Head 1	Head 2	Head 3
Who does IT mean?	Grammar: who does what?	Which word describes which?
Q · K · V	Q · K · V	Q · K · V

Feed-forward = desk time
Each word thinks it over on its own

Inside every head: Q · K · V

Query (Q) = my question
"What am I looking for?"

Key (K) = my name tag
"Here's what I am."

Value (V) = my info card
"Here's what I share."

How one head works
Match my Q against every K → pick who to listen to → blend their V into my new meaning

Copyright © 2026 Nanyang Technological University. All rights reserved.

Same Recipe, Different Scale

From MiniLM to GPT-4

Model	Params (log scale)	Layers	d_model	Training data
all-MiniLM-L6-v2 (L09)	22M	6	384	1B+ sentence pairs
BERT-base	110M	12	768	Wikipedia + books
GPT-2 medium	355M	24	1,024	WebText (40GB)
SmolLM2-360M (today's LLM)	361M	32	960	Open web + curation
GPT-3	175B	96	12,288	Internet + books + code
GPT-4 (estimated)	~1T	?	?	Internet + code + RLHF

Same Lego brick. What changes: more bricks, more data, more post-training, more money.

Copyright © 2026 Nanyang Technological University. All rights reserved.

CODE-ALONG

Open: 02_attention_intuition.ipynb

What you'll do

- Watch a real model decide who “it” and “she” listen to
- See the answer form layer by layer; read a heatmap
- See multi-head attention with two patterns side by side
- Sketch a transformer block in PyTorch

25 minutes · Stop at the Extension marker

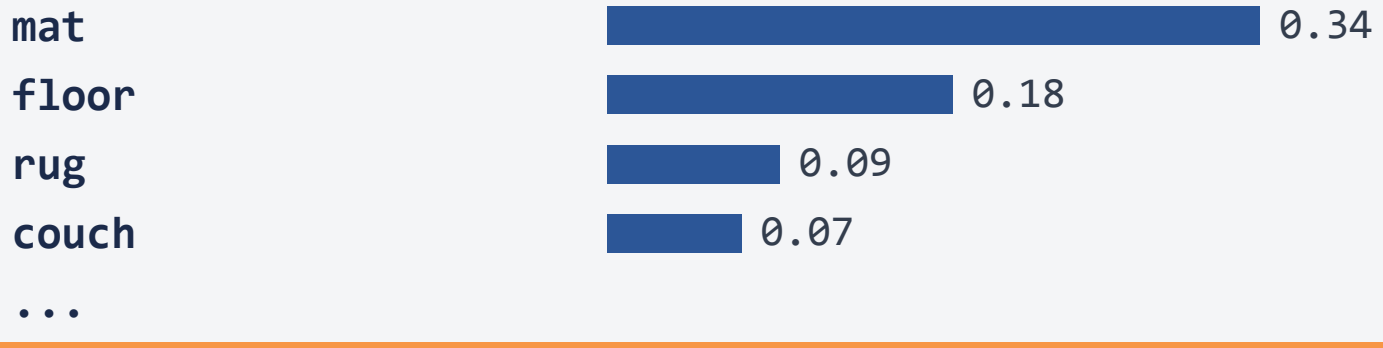
What An LLM Actually Does

Predict the next token. That's it.

Input

"The cat sat on the"

→ Probability distribution over the vocabulary:



Predict probabilities → **Pick one token** → **Append it to the text** → **Run again ↻**

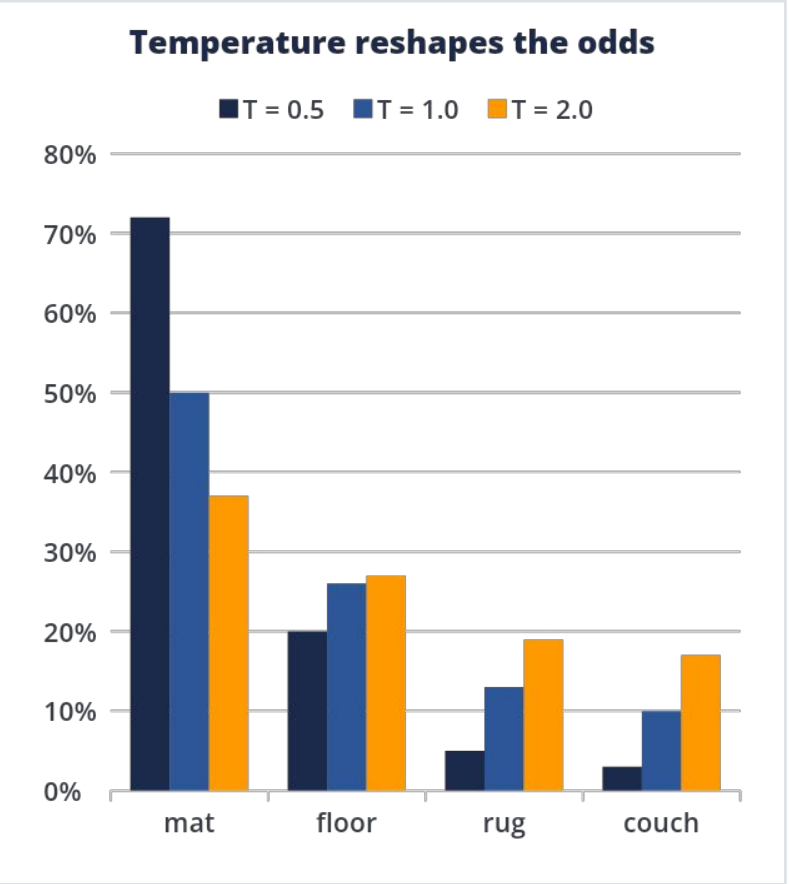
Every ChatGPT response is generated by this loop, one token at a time.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Sampling — From Distribution To Token

Temperature, top-k, top-p

Greedy	Pick argmax. Deterministic, often repetitive. Good for code.
Temperature	Scale logits before softmax. T<1 → sharper; T>1 → flatter. T=0 ≡ greedy.
Top-k	Only sample from the k highest-probability tokens. Cuts the long tail.
Top-p (nucleus)	Only sample from the smallest set summing to probability mass p. Adaptive cutoff.



Production default: T=0.7 + top-p=0.9. Lower for code & facts (T=0.2). Higher for creative writing (T=1.0+).

Copyright © 2026 Nanyang Technological University. All rights reserved.

Chat Templates

Format matters — instruction-tuned models expect specific tokens

SmolLM2's chat template (each model family has its own):

```
<|im_start|>system
You are a helpful retail assistant.<|im_end|>
<|im_start|>user
What's a good summer dress?<|im_end|>
<|im_start|>assistant
```

Three rules

- Use tokenizer.apply_chat_template() — never hand-roll the format
- system → user → assistant turn order (or just user → assistant)
- Get the format wrong and the model behaves like garbage

Copyright © 2026 Nanyang Technological University. All rights reserved.

CODE-ALONG

Open: 03_using_an_llm.ipynb

What you'll do

- Load SmolLM2-360M-Instruct from Hugging Face
- Watch tokenisation: text → IDs → logits → next token
- Manual generation loop (10 lines) and model.generate()
- Sampling: greedy vs T=0.7 vs T=1.5; chat templates

30 minutes · Stop at the Extension marker

Hallucination

The LLM doesn't know YOUR data — so it invents

Q: "I'm looking for a summer dress under £80. What does NorthStar have?"

```
A (LLM alone): "NorthStar has several stylish and affordable options... Luxury Dress...
Midi Dress... Prices vary depending on the brand and style."
```

NORTHSTAR SELLS NO “MIDI DRESS” LINE.

The model invented a plausible-sounding product. Fluent. Confident. Completely fictional.
Customers who click and find nothing lose trust forever.

Why this happens

The LLM was trained to produce plausible text. It has no access to ground truth.
For any factual question about YOUR data, it will improvise. The fix: give it the data.

Copyright © 2026 Nanyang Technological University. All rights reserved.

RAG — The Fix

Retrieve · Augment · Generate

RETRIEVE | **AUGMENT** | **GENERATE**

1. **Customer asks**
   "summer dress under £80"
2. **Embed the query**
   Turn the question into a vector with the L09 model
3. **Find the top 5**
   Cosine similarity over every catalogue product; keep the 5 closest
4. **Augment the prompt**
   Paste the 5 products into the USER message; the SYSTEM prompt says use ONLY these
5. **Generate**
   LLM writes the answer, grounded in real products

Step 4 in practice: the USER message

```
QUESTION: summer dress under £80
PRODUCTS: <the 5 retrieved products>
Recommend at most two products from the list above.
```

Why it works

Like an open-book exam: the LLM can only pick from real products, so no invented "Midi Dress".

Copyright © 2026 Nanyang Technological University. All rights reserved.

Why RAG Works

Each side fixes the other's weakness

The LLM brings

- Language fluency (formulating sentences)
- General reasoning (matching needs to products)
- Question understanding (intent extraction)
- Stylistic flexibility (concise / detailed)

The retrieval brings

- Up-to-date factual data
- Specific products the LLM doesn't know
- Audit trail (you can show source documents)
- Easy update (re-embed; no retraining)

LLM
fluent, but doesn't know your products

+

Retrieval
factual, but can't hold a conversation

=

RAG
fluent AND grounded in real data

Copyright © 2026 Nanyang Technological University. All rights reserved.

CODE-ALONG

Open: 04_rag_pipeline.ipynb

What you'll do

- Show LLM hallucinating without context — be horrified
- Build the retrieve() function (cosine over catalogue embeddings)
- Construct the augmented prompt (products in the user turn)
- Wrap it in a RAGSystem class — production-ready ~60 lines

35 minutes · Stop at the Extension marker

Sarah's Recommendation

Production-grade RAG

Sarah, end of L10:

"RAG over the catalogue. all-MiniLM-L6-v2 for retrieval. SmolLM2-360M for the demo. For production: hosted API (Claude Haiku / GPT-4 mini) for quality + cost. Always pass top-5 docs + a strict system prompt."

Production guardrails (non-negotiable)

- System prompt: "ONLY recommend products from the catalogue below."
- Validate product names in the answer against the retrieved set (catch hallucinations).
- Show source products alongside the answer (audit trail).
- Log (query → retrieved → answer) for offline eval and weekly quality review.
- A/B test against L09 keyword+semantic baseline before going broad.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Deployment Trade-Off

Cost · Latency · Quality — pick two

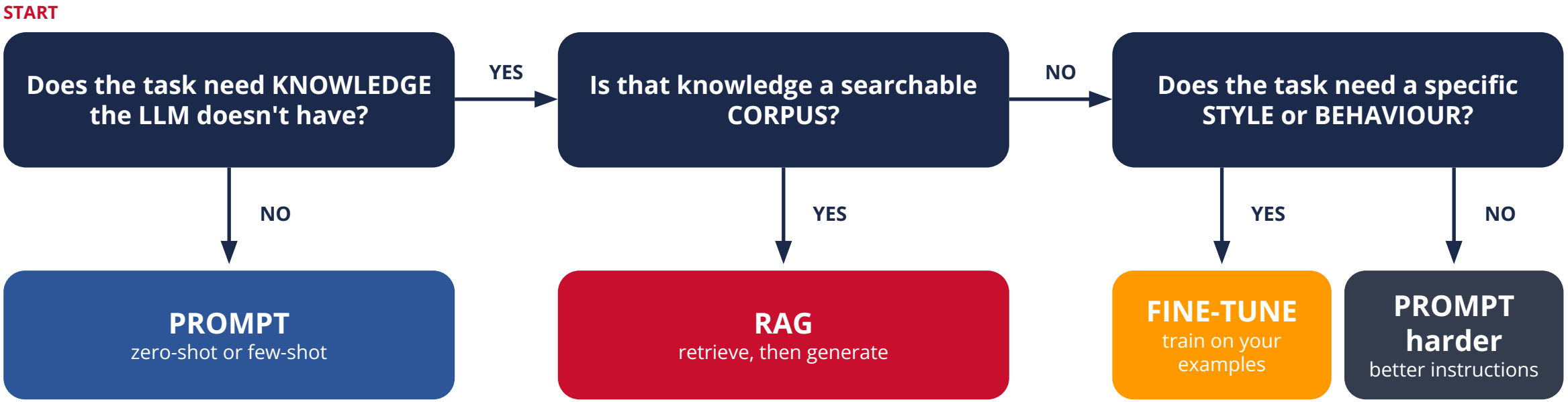
Option	Latency	Quality / cost
Self-host SmolLM2 (free)	`~5-15 s/response on a laptop`	Decent for demo, poor for prod
Self-host 7B on GPU	`~200 ms`	Good. Infra: $$$/month
Hosted Claude Haiku / GPT-4 mini	`~500 ms`	Excellent. ~$0.001 per query
Hosted GPT-4 / Claude Sonnet	`~2 s`	Best in class. ~$0.03 per query
Fine-tuned in-house model	`~100-500 ms`	Domain-tuned; high setup cost

Most production teams start with hosted API + RAG. Self-host or fine-tune later if needed.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Decision Tree

Prompt vs RAG vs fine-tune



e.g. sentiment, summarisation, format conversion

e.g. product catalogue, internal docs, code search

e.g. medical reports, brand voice, structured output

Rule of thumb: try the cheapest option first. Prompt → RAG → fine-tune.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Three Things To Remember

L10 takeaways

1

Attention is the mechanism

Every modern NLP model — embeddings, classifiers, LLMs — is a stack of transformer blocks. The recipe doesn't change with scale.

2

An LLM is a next-token predictor

That's literally all it does. Sampling turns a probability distribution into text. Generation is a one-token-at-a-time loop.

3

RAG is the standard for YOUR data

Retrieve, augment, generate. Don't expect the LLM to know your catalogue — give it the catalogue. Guard against hallucination with strict system prompts.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Method Cheat Sheet

GenAI tools in production

Scenario	Recommendation
Sentiment / NER / classification	**Pretrained pipeline (transformers.pipeline)**
Chat / Q&A about your data	**RAG: embedding retrieval + LLM generation**
Highly specific domain (medical, legal)	**Start with RAG; fine-tune if quality insufficient**
Style transfer / brand voice	**Fine-tune a small base model on examples**
Experimental / one-off	**Hosted API (Claude, GPT). Pay-per-use.**
Cost-sensitive / high volume	**Self-hosted small model or fine-tuned own**
Safety-critical	**Human-in-the-loop. LLMs hallucinate.**

Copyright © 2026 Nanyang Technological University. All rights reserved.

Common Pitfalls

What goes wrong in production GenAI

1. **Trusting the LLM blindly**
   It will hallucinate confidently. Always validate against ground truth.
2. **Hand-rolling chat formats**
   Use tokenizer.apply_chat_template(). Wrong format = garbage output.
3. **Wrong temperature for the task**
   T=0 for code/facts. T=0.7 for chat. T=1+ for creative writing.
4. **Skipping the system prompt**
   System prompt sets behaviour. "Only use the catalogue below" prevents hallucinations.
5. **Fine-tuning prematurely**
   Try prompt + few-shot + RAG first. Fine-tuning has high setup cost.
6. **Ignoring evaluation**
   Run a hand-graded benchmark weekly. Quality drifts; you won't notice without measurement.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Module 3 — The Toolkit You've Built

Ten lessons, one journey

L01-L02	**Classical ML**	LR, classification metrics, model evaluation
L03	**Making it work**	Pipelines, calibration, threshold tuning, fairness
L04	**Trees + tuning**	Gradient boosting, hyperparameter search
L05	**Unsupervised**	Clustering, PCA, anomaly detection
L06	**Time series**	Forecasting, holiday effects, evaluation
L07	**Neural networks**	MLPs, PyTorch, the training loop
L08	**Computer vision**	CNNs, transfer learning, ResNet18
L09	**NLP & embeddings**	Semantic search, hybrid retrieval
L10	**Transformers & GenAI**	Attention, LLMs, RAG

You can ship classical ML, deep learning, semantic search, AND GenAI. End-to-end.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Sarah's Journey — And Yours

What the narrative was teaching

Each lesson was a tool. The story was about how to choose.

- L01-L03: ship the simplest model first; measure it; improve only what fails
- L04-L06: pick the tool for the data — trees for tabular, clustering for discovery, ARIMA for seasonal
- L07-L08: the same training loop works for any architecture; the data and structure are what matter
- L09: embeddings expose meaning as geometry; combine semantic with classical for production search
- L10: GenAI is real but hallucinates; RAG keeps it grounded; you can ship without fine-tuning

The mathematics is the easy part. Picking the right tool is taste — and you've started developing it.

Copyright © 2026 Nanyang Technological University. All rights reserved.

What's Next

Directions for further study

Domain specialisation	Pick a vertical — healthcare, finance, retail, NLP, vision — and go deep. Become the person at your company who knows that area.
MLOps & evaluation	Shipping is one thing. Keeping a model working over months is harder. Learn evaluation, monitoring, A/B testing, drift detection.
Fine-tuning & alignment	PEFT, LoRA, RLHF, DPO. How to bend pretrained models to your specific use case. Mostly only worth learning when you have a real task that needs it.
Read the papers	Attention Is All You Need, BERT, GPT-2/3/4, CLIP, LLaMA, DPO. Reading is faster than waiting for the next course.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Final Assignment

Ship a RAG shopping assistant

Assignment — pick ONE track

Foundational track: Part A — implement a RAGSystem class (retrieve + ask)
Advanced track: Part B — run a 5-query benchmark; grade each on 3 criteria
+ Part C — identify two specific weaknesses + concrete fixes

Optional self-study

- optional_extensions.ipynb — prompt engineering, sampling deep-dive, decision tree
- Try a hosted API (Claude / OpenAI) in place of SmolLM2 — compare quality
- Add the hallucination-check pattern from NB 04 Extensions

This is the LAST assignment of the module. Submit by end of next week.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Resources To Keep Building

Books, papers, communities, courses

Type	Recommendation
Books	Murphy, Bishop, Hastie/Tibshirani (classical); Goodfellow, Bishop (DL); HF book
Papers	arxiv.org/list/cs.LG — Karpathy's reading list; Yannic Kilcher's YouTube
Courses	Fast.ai (practical DL); DeepLearning.AI; Stanford CS224N (NLP), CS231N (CV)
Communities	r/MachineLearning; HuggingFace forums; Kaggle competitions; arxiv-sanity
Build	Spend more time shipping than studying. Build something every week.

If you only do ONE thing: build a side project. Anything. Ship it. Then iterate.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Thank you

for nine weeks of curiosity, debugging, honest pushback, and shipping

You now have the modern ML toolkit.

Use it well. Be honest about what works.

Now go build something.